

Random Forest implementation from scratch

Objective

You are required to implement a Random Forest classifier **from scratch** to predict passenger survival on the Titanic dataset. Your implementation should replicate the exact data processing, train/test split, and Random Forest logic as described below. Ensure reproducibility of results using fixed random seeds.

Dataset Input

1. The dataset will be provided in **space-separated text format** with the first line as the header.
2. Input format (for standard input):
 - First line: Number of rows n (including the header).
 - Second line: Header (column names) separated by spaces.
 - Next $n - 1$ lines: Data rows with values separated by spaces.
3. **Important Columns:** Keep only the following columns for modeling (ignore the rest if present):

Pclass, Sex, Age, SibSp, Parch, Fare, Embarked, Survived

4. **Numeric Columns:** Convert Pclass, Age, SibSp, Parch, Fare, Survived to numeric.
5. **Categorical Encoding:** Encode Sex and Embarked as integers starting from 0.

Randomization Seeds

- Python: `random.seed(42)`
- Numpy: `np.random.seed(42)`
- Data shuffling for train/test split: `random_state=217`

Train/Test Split

1. Shuffle the dataset randomly (`frac=1`) with `random_state=217`.
2. Use 80% of the data for training and 20% for testing.
3. Features for modeling:

['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']

4. Target variable: `Survived`.

Random Forest Implementation

Hyperparameters

- `n_estimators` = 100 (number of trees)
- `max_features` = 3 (max features to consider when splitting)
- `max_depth` = 10 (max depth of each tree)
- `min_samples_split` = 2 (min samples to split a node)

Steps

1. **Bootstrap Sampling:** Randomly sample with replacement from training data for each tree. The remaining data are called Out-Of-Bag (OOB) samples.
2. **Splitting Nodes:**
 - Select `max_features` random features.
 - For each feature, try all unique values as potential split points.
 - Split according to:
 - Continuous: $value \leq split_point \rightarrow left$, else $right$
 - Categorical: $value = split_point \rightarrow left$, else $right$
3. **Entropy:**

$$\text{Entropy}(p) = -(p \log_2 p + (1 - p) \log_2 (1 - p)), \quad p \in (0, 1)$$

4. **Information Gain:**

$$IG(\text{split}) = \text{Entropy}(\text{parent}) - \frac{|L|}{|P|} \text{Entropy}(L) - \frac{|R|}{|P|} \text{Entropy}(R)$$

5. **Terminal Node:** If a node cannot be split further (empty child, depth limit, or min samples), assign the majority class as the node value.
6. **Recursive Splitting:** Continue splitting left and right children recursively until *max_depth* or *min_samples_split* is reached.
7. **Tree Prediction:** Traverse each tree comparing feature values at each split to determine the class.
8. **Random Forest Prediction:** Aggregate predictions from all trees using **majority vote**.
9. **OOB Score (optional):** Compute misclassification rate on OOB samples for each tree.

Expected Output

1. OOB Estimate: Mean OOB misclassification across all trees.
2. Testing Accuracy: Accuracy on the test set after majority voting.

Instructions to Students

1. Use Python 3 with libraries: `pandas`, `numpy`, `random`, `sys`. Do not use `sklearn` or other ML libraries.
2. Fix random seeds as mentioned to ensure reproducibility.
3. Read input exactly as specified (simulate Moodle-style stdin input).
4. Implement all functions yourself: `entropy`, `information_gain`, `draw_bootstrap`, `find_split_point`, `split_node`, `build_tree`, `random_forest`, `predict_tree`, `predict_rf`.
5. Ensure that the order of train/test split and categorical encoding is consistent.

Key Formulas

Formula	Description
Entropy	$-(p \log_2 p + (1 - p) \log_2 (1 - p))$
Information Gain	$IG = \text{Entropy}(\text{parent}) - \frac{ L }{ P } \text{Entropy}(L) - \frac{ R }{ P } \text{Entropy}(R)$
Majority Class	Node class = class with highest count in node samples

Deliverables

1. Python script implementing Random Forest as described.
2. Output displaying:
 - OOB estimate
 - Test accuracy

Reproducibility Check: Using the fixed random seeds, multiple runs of the code should produce identical OOB estimate and test accuracy.